p0907r0
Signed Integers are Two's Complement

Published Proposal, 9 February 2018

This version:
http://wg21.link/P0907r0

Author:
JF Bastien (Apple)

Audience:
EWG, SG12

Toggle Diffs:
☐ Hide deleted text

Project:
ISO JTC1/SC22/WG21: Programming Language C++

Source:
github.com/jfbastien/papers/blob/master/source/P0907r0.bs

Abstract

There is One True Representation for signed integers, and that representation is two's complement.

Table of Contents

References

Informative References

§ 1. Introduction

[C11] Integer types allows three representations for signed integral types:

- Signed magnitude
- Ones' complement
- Two’s complement

See §4 C Signed Integer Wording for full wording.

C++ inherits these three signed integer representations from C. To the author’s knowledge no modern machine uses both C++ and a signed integer representation other than two’s complement (see §5 Survey of Signed Integer Representations). None of [MSVC], [GCC], and [LLVM] support other representations. This means that the C++ that is taught is effectively two’s complement, and the C++ that is written is two’s complement. It is extremely unlikely that there exist any significant code base developed for two’s complement machines that would actually work when run on a non-two’s complement machine.

The C++ that is spec’d, however, is not two’s complement. Signed integers currently allow for trap representations, extra padding bits, integral negative zero, and introduce undefined behavior and implementation-defined behavior for the sake of this *extremely* abstract machine.

Specifically, the current wording has the following effects:

- Associativity and commutativity of integers is needlessly obtuse.
- Naïve overflow checks, which are often security-critical, often get eliminated by compilers. This leads to exploitable code when the intent was clearly not to and the code, while naïve, was correctly performing security checks for two’s complement integers. Correct overflow checks are difficult to write and equally difficult to read, exponentially so in generic code.
- Conversion between signed and unsigned are implementation-defined.
- There is no portable way to generate an arithmetic right-shift, or to sign-extend an integer, which every modern CPU supports.
- `constexpr` is further restrained by this extraneous undefined behavior.
- Atomic integral are already two’s complement and have no undefined results, therefore even freestanding implementations already support two’s complement in C++.

Let’s stop pretending that the C++ abstract machine should represent integers as signed magnitude or ones' complement. These theoretical implementations are a different programming language, not our real-world C++. Users of C++ who require signed magnitude or ones' complement integers would be better served by a pure-library solution, and so would the rest of us.

This proposal leaves C unchanged, it merely restricts further the subset of C which applies to C++. The author will ensure that WG14 is made aware of this paper’s outcome.

A final argument to move to two’s complement is that few people spell "ones' complement" correctly according to Knuth [TAoCP]. Reducing the nerd-snipe potential inherent in C++ is a Good Thing™.

> Detail-oriented readers and copy editors should notice the position of the apostrophe in terms like “two’s complement” and “ones’ complement”: A two’s complement number is complemented with respect to a single power of 2, while a ones’ complement number is complemented with respect to a long sequence of 1s. Indeed, there is also a “twos’ complement notation,” which has radix 3 and complementation with respect to $(2 \ldots 22)_3$.

§ 2. Proposed Wording

Modify Program execution [**intro.execution**] ¶8:

[*Note:* Operators can be regrouped according to the usual mathematical rules only where the operators really are associative or commutative. For example, in the following fragment

```
int a, b;
/* ... */
a = a + 32760 + b + 5;
```

the expression statement behaves exactly the same as

```
a = (((a + 32760) + b) + 5);
```

due to the associativity and precedence of these operators. Thus, the result of the sum `(a + 32760)` is next added to `b`, and that result is then added to 5 which results in the value assigned to `a`. ~~On a machine in which overflows produce an exception and in which the range of values representable by an `int` is `[-32768, +32767]`, the implementation cannot rewrite this expression as~~

```
~~a = ((a + b) + 32765);~~
```

~~since if the values for `a` and `b` were, respectively, -32754 and -15, the sum `a + b` would produce an exception while the original expression would not; nor can the expression be rewritten either as~~

```
~~a = ((a + 32765) + b);~~
```

~~or~~

```
~~a = (a + (b + 32765));~~
```

~~since the values for `a` and `b` might have been, respectively, 4 and -8 or -17 and 12. However on a machine in which overflows do not produce an exception and in which the results of overflows are reversible, the above expression statement can be rewritten by the implementation in any of the above ways because the same result will occur.~~ —*end note*]

Modify Fundamental types [**basic.fundamental**] ¶4 onwards:

> Unsigned integers shall obey the laws of arithmetic modulo 2^n where *n* is the number of bits in the value representation of that particular size of integer.
>
> > This implies that unsigned arithmetic does not overflow because a result that cannot be represented by the resulting unsigned integer type is reduced modulo the number that is one greater than the largest value that can be represented by the resulting unsigned integer type.
>
> Type `wchar_t` is a distinct type whose values can represent distinct codes for all members of the largest extended character set specified among the supported locales. Type `wchar_t` shall have the same size, signedness, and alignment requirements as one of the other integral types, called its *underlying type*. Types `char16_t` and `char32_t` denote distinct types with the same size, signedness, and alignment as `uint_least16_t` and `uint_least32_t`, respectively, in `<cstdint>`, called the underlying types.
>
> Values of type `bool` are either `true` or `false`. [*Note:* There are no `signed`, `unsigned`, `short`, or `long bool` types or values. —*end note*] Values of type `bool` participate in integral promotions.
>
> Types `bool`, `char`, `char16_t`, `char32_t`, `wchar_t`, and the signed and unsigned integer types are collectively called *integral* types. A synonym for integral type is *integer type*. ~~The representations of integral types shall define values by use of a pure binary numeration system. [*Example:* This document permits two's complement, ones' complement and signed magnitude representations for integral types. —*end example*]~~
>
> Signed integer types shall be repesented as two's complement. Overflow in the positive direction shall wrap around from the maximum integer value for the type back to the minimum, and overflow in the negative direction shall wrap around from the minimum value for the type to the maximum. [*Note:* Addition, subtraction, and multiplication on signed and unsigned integral values with the same object representation produce a value with the same object representation, whereas division and modulo do not. —*end note*]

Modify Integral conversions [**conv.integral**] ¶1 onwards:

> A prvalue of an integer type can be converted to a prvalue of another integer type. A prvalue of an unscoped enumeration type can be converted to a prvalue of an integer type.
>
> If the destination type is unsigned, the resulting value is the least unsigned integer congruent to the source integer (modulo 2^n where *n* is the number of bits used to represent the unsigned type). ~~[*Note:* In a two's complement representation, t~~ T his conversion is conceptual and there is no change in the bit pattern (if there is no truncation). ~~— *end note*]~~
>
> If the destination type is signed, the value is unchanged if it can be represented in the destination type; otherwise, ~~the value is implementation-defined.~~ the object representation remains the same if the source and destination have the same size, or the least-significant source bits are retained if the destination is smaller than the source.

Modify Static cast [**expr.static.cast**] ¶1 onwards:

> A value of a scoped enumeration type can be explicitly converted to an integral type. When that type is *cv* `bool`, the resulting value is `false` if the original value is zero and `true` for all other values. For the remaining integral types, the value is unchanged if the original value can be represented by the specified type. Otherwise, the ~~resulting value is unspecified~~ the object representation remains the same if the source and destination have the same size, or the least-significant source bits are retained if the destination is smaller than the source . A value of a scoped enumeration type can also be explicitly converted to a floating-point type; the result is the same as that of converting from the original value to the floating-point type.
>
> A value of integral or enumeration type can be explicitly converted to a complete enumeration type. If the enumeration type has a fixed underlying type, the value is first converted to that type by integral conversion, if necessary, and then to the enumeration type. If the enumeration type does not have a fixed underlying type, the value is unchanged if the original value is within the range of the enumeration values, and otherwise, ~~the behavior is undefined~~ the object representation remains the same if the source and destination have the same size, or the least-significant source bits are retained if the destination is smaller than the source . A value of floating-point type can also be explicitly converted to an enumeration type. The resulting value is the same as converting the original value to the underlying type of the enumeration\iref{conv.fpint}, and subsequently to the enumeration type.

Modify Shift operators [**expr.shift**] ¶1 onwards:

> The operands shall be of integral or unscoped enumeration type and integral promotions are performed. The type of the result is that of the promoted left operand. The behavior is undefined if the right operand is negative, or greater than or equal to the length in bits of the promoted left operand.
>
> The value of `E1 << E2` is `E1` left-shifted `E2` bit positions; vacated bits are zero-filled. ~~If `E1` has an unsigned type, t~~ T he value of the result is $E1 \times 2^{E2}$, reduced modulo one more than the maximum value representable in the result type. ~~Otherwise, if `E1` has a signed type and non-negative value, and $E1 \times 2^{E2}$ is representable in the corresponding unsigned type of the result type, then that value, converted to the result type, is the resulting value; otherwise, the behavior is undefined.~~
>
> The value of `E1 >> E2` is `E1` right-shifted `E2` bit positions. If `E1` has an unsigned type or if `E1` has a signed type and a non-negative value, the value of the result is the integral part of the quotient of $E1/2^{E2}$. If `E1` has a signed type and a negative value, the resulting value ~~is implementation-defined.~~ the negative of the integral part of the quotient of $E1/2^{E2}$. [*Note*: This implies that right-shift on signed integral types is an arithmetic right shift, and performs sign-extension. —*end note*]

Modify Constant expressions [**expr.const**] ¶2:

> An expression `e` is a *core constant expression* unless the evaluation of `e`, following the rules of the abstract machine, would evaluate one of the following expressions:
>
> […]
>
> - an operation that would have undefined behavior as specified in Clause 4 through 19 of this document [*Note*: including, for example, ~~signed integer overflow,~~ certain pointer arithmetic, division by zero, or certain shift operations —*end note*]

Modify Enumeration declarations [**dcl.enum**] ¶8:

> For an enumeration whose underlying type is fixed, the values of the enumeration are the values of the underlying type. Otherwise, for an enumeration where e_{min} is the smallest enumerator and e_{max} is the largest, the values of the enumeration are the values in the range b_{min} to b_{max}, defined as follows: ~~Let *K* be 1 for a two's complement representation and 0 for a ones' complement or sign-magnitude representation.~~ b_{max} is the smallest value greater than or equal to max($|e_{min}|$ - ~~K~~ 1 , $|e_{max}|$) and equal to 2^M-1, where *M* is a non-negative integer. b_{min} is zero if e_{min} is non-negative and -(b_{max}+ ~~K~~ 1) otherwise. The size of the smallest bit-field large enough to hold all the values of the enumeration type is max(M,1) if b_{min} is zero and M+1 otherwise. It is possible to define an enumeration that has values not defined by any of its enumerators. If the *enumerator-list* is empty, the values of the enumeration are as if the enumeration had a single enumerator with value 0.

Modify `numeric_limits` members [**numeric.limits.members**] ¶61 onwards:

> ```
> static constexpr bool is_modulo;
> ```
>
> `true` if the type is modulo. A type is modulo if, for any operation involving `+`, `-`, or `*` on values of that type whose result would fall outside the range `[min(), max()]`, the value returned differs from the true value by an integer multiple of `max() - min() + 1`.
>
> ~~[*Example:* `is_modulo` is `false` for signed integer types unless an implementation, as an extension to this document, defines signed integer overflow to wrap. —*end example*]~~
>
> Meaningful for all specializations.

Modify Class template `ratio` [**ratio.ratio**] ¶1:

> If the template argument `D` is zero or the absolute values of either of the template arguments `N` and `D` is not representable by type `intmax_t`, the program is ill-formed. [*Note:* These rules ensure that infinite ratios are avoided and that for any negative input, there exists a representable value of its absolute value which is positive. ~~In a two's complement representation, t~~ T his excludes the most negative value. —*end note*]

Remove Specializations for integers [**atomics.types.int**] ¶7:

> ~~*Remarks:* For signed integer types, arithmetic is defined to use two's complement representation. There are no undefined results.~~

§ 3. Out of Scope

This proposal focuses on the representation of signed integers, and on tightening the specification when that representation is constrained to two's complement. It is out of scope for this proposal to deal with related issues which have more to them than simply the representation of signed integers.

A non-comprehensive list of items left purposefully out:

- Left and right shift with a right-hand-side equal to or wider than the bit-width of the left-hand-side.
- Integral division or modulo by zero.
- Integral division or modulo of the signed minimum integral value for a particular integral type by minus one.
- Overflow of pointer arithmetic.
- Library solution for ones' complement integers.
- Library solution for signed magnitude integers.
- Library solution for two's complement integers with trapping or undefined overflow sepantics.
- Language support for explicit signed overflow truncation such as Swift's (`&+`, `&-`, and `&*`), or complementary trapping overflow operators.
- Library or language support for saturating arithmetic.
- Mechanism to let the compiler assume that integers, signed or unsigned, do not experience signed or unsigned wrapping for:
  - A specific integral variable.
  - All integral variables (à la `-ftrapv`, `-fno-wrapv`, and `-fstrict-overflow`).
  - A specific loop's induction variable.
- Mechanism to have the compiler list places where it could benefit from knowing that overflow cannot occur (à la `-Wstrict-overflow`).
- Endianness of integral storage (or endianness in general).
- Bits per bytes, though we all know there are eight.

These items could be tackled in separate proposals, unless the committee wants them tackled here. This paper expresses no preference in whether they should be addressed or how.

§ 4. C Signed Integer Wording

The following is the wording on integers from the C11 Standard.

For unsigned integer types other than unsigned char, the bits of the object representation shall be divided into two groups: value bits and padding bits (there need not be any of the latter). If there are *N* value bits, each bit shall represent a different power of 2 between 1 and 2^{N-1}, so that objects of that type shall be capable of representing values from 0 to $2^{N} - 1$ using a pure binary representation; this shall be known as the value representation. The values of any padding bits are unspecified.

For signed integer types, the bits of the object representation shall be divided into three groups: value bits, padding bits, and the sign bit. There need not be any padding bits; `signed char` shall not have any padding bits. There shall be exactly one sign bit. Each bit that is a value bit shall have the same value as the same bit in the object representation of the corresponding unsigned type (if there are *M* value bits in the signed type and *N* in the unsigned type, then M ≤ N). If the sign bit is zero, it shall not affect the resulting value. If the sign bit is one, the value shall be modified in one of the following ways:

- the corresponding value with sign bit 0 is negated (*sign and magnitude*);
- the sign bit has the value $-(2^{M})$ (*two's complement*);
- the sign bit has the value $-(2^{M} - 1)$ (*ones' complement*).

Which of these applies is implementation-defined, as is whether the value with sign bit 1 and all value bits zero (for the first two), or with sign bit and all value bits 1 (for ones' complement), is a trap representation or a normal value. In the case of sign and magnitude and ones' complement, if this representation is a normal value it is called a *negative zero*.

If the implementation supports negative zeros, they shall be generated only by:

- the `&`, `|`, `^`, `~`, `<<`, and `>>` operators with operands that produce such a value;
- the `+`, `-`, `*`, `/`, and `%` operators where one operand is a negative zero and the result is zero;
- compound assignment operators based on the above cases.

It is unspecified whether these cases actually generate a negative zero or a normal zero, and whether a negative zero becomes a normal zero when stored in an object.

If the implementation does not support negative zeros, the behavior of the `&`, `|`, `^`, `~`, `<<`, and `>>` operators with operands that would produce such a value is undefined.

The values of any padding bits are unspecified. A valid (non-trap) object representation of a signed integer type where the sign bit is zero is a valid object representation of the corresponding

> unsigned type, and shall represent the same value. For any integer type, the object representation where all the bits are zero shall be a representation of the value zero in that type.
>
> The *precision* of an integer type is the number of bits it uses to represent values, excluding any sign and padding bits. The *width* of an integer type is the same but including any sign bit; thus for unsigned integer types the two values are the same, while for signed integer types the width is one greater than the precision.

§ 5. Survey of Signed Integer Representations

Here is a non-comprehensive history of signed integer representations:

- Two’s complement
  - John von Neumann suggested use of two’s complement binary representation in his 1945 First Draft of a Report on the EDVAC proposal for an electronic stored-program digital computer.
  - The 1949 EDSAC, which was inspired by the First Draft, used two’s complement representation of binary numbers.
  - Early commercial two’s complement computers include the Digital Equipment Corporation PDP-5 and the 1963 PDP-6.
  - The System/360, introduced in 1964 by IBM, then the dominant player in the computer industry, made two’s complement the most widely used binary representation in the computer industry.
  - The first minicomputer, the PDP-8 introduced in 1965, uses two’s complement arithmetic as do the 1969 Data General Nova, the 1970 PDP-11.
- Ones' complement
  - Many early computers, including the CDC 6600, the LINC, the PDP-1, and the UNIVAC 1107.
  - Successors of the CDC 6600 continued to use ones' complement until the late 1980s.
  - Descendants of the UNIVAC 1107, the UNIVAC 1100/2200 series, continue to do so, although ClearPath machines are a common platform that implement either the 1100/2200 architecture (the ClearPath IX series) or the Burroughs large systems architecture (the ClearPath NX series). Everything is common except the actual CPUs, which are implemented as ASICs. In addition to the IX (1100/2200) CPUs and the NX (Burroughs large systems) CPU, the architecture had Xeon (and briefly Itanium) CPUs. Unisys' goal was

to provide an orderly transition for their 1100/2200 customers to a more modern architecture.

- Signed magnitude
  - The IBM 700/7000 series scientific machines use sign/magnitude notation, except for the index registers which are two's complement.

Wikipedia offers more details and has comprehensive sources for the above.

In short, the only machine the author could find using non-two's complement are made by Unisys. Nowadays they emulate their old architecture using x86 CPUs for customers who have legacy applications which they've been unable to migrate. These applications are unlikely to be well served by modern C++, signed integers are the least of their problem. Post-modern C++ should focus on serving its existing users well, and incoming users should be blissfully unaware of integer esoterica.

§ References

§ Informative References

[C11]
Programming Languages — C. URL: http://www.open-std.org/jtc1/sc22/wg14/www/docs/n1570.pdf

[GCC]
GCC C Implementation-Defined Behavior: Integers. URL: https://gcc.gnu.org/onlinedocs/gcc/Integers-implementation.html

[LLVM]
LLVM Language Reference Manual. URL: https://llvm.org/docs/LangRef.html

[MSVC]
MSVC C Implementation-Defined Behavior: Integers. URL: https://docs.microsoft.com/en-us/cpp/c-language/integers

[TAoCP]
Donald Knuth. The Art of Computer Programming, Volume 2 (3rd Ed.): Seminumerical Algorithms.